

Chapter 3

Hyperparameters

1. Epochs

- Why multiple epochs? a model can't learn from seeing the data once just like a human
- Underfitting: Underperforming (train for 2 epochs) or having very poor hyperparameters
- Overfitting: Overperforming (train for 10000 epochs) the model memorizes the training data, it will perform flawlessly on training set & fail miserably on new data.
- Early stopping: Instead of guessing the number of epochs, we set it at a high number (e.g 300) while monitoring validation metrics
→ if they aren't improving for say 20 epochs we stop training.

2. Batch Size

- Memory Constraints: GPU has limited VRAM (e.g 16GB) if the images are large we can only fit a small batch size (e.g 8) at once.
- Gradient Estimation

	Computationally impossible	Noisy updates
⊗ Mini Batch ⊗ Batch of 8, 16, 32... Most balanced requires tuning of batch size	⊗ Batch GD ⊗ Whole dataset in one batch. Highly accurate but requires massive memory	⊗ Stochastic GD ⊗ Batch size is 1 → update weights with every image. Escapes local minima Slow per epoch & doesn't use GPU vectorization

3. Learning Rate (α)

Most critical hyperparameter. The step size to reach the lowest loss possible.

- too large: Overshoots over the goal, bouncing back & forth
- too small: Tiny learning speed that will take an eternity to reach the goal or even get stuck in a local minima

⊛ Schedulers & Warmups ⊛ We don't keep LR constant

⊛ Warmup: Start with tiny LR & gradually increase it, prevents the model from making bad, massive weight changes in the very first few batches.

⊛ Cosine Decay: As training progresses & we get closer to the optimal loss, we gradually shrink LR so the model can fine-tune its position without overshooting.

4. Image Size

Why resize 4K image down to 640 x 640 e.g.?

- Memory & Speed
- Receptive Fields → feeding a 4K image the model has to be way deeper to get a receptive field of the full context at the deep layers.
- Small Object Detection: Distant objects (drones, small person...) might be compressed down to a single pixel & get lost entirely.

We keep this in mind if we have small objects.